Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Gruppe 2

Haus Slytherin

Teil 1

Slytherin Software Engineering Training

Story:

Das **Slytherin Magical Systems Lab** analysiert Software für Hogwarts.
Ihr bekommt u.a. **unfertige oder schlecht designte Systeme** und müsst sie verbessern.

Git Regel

Nach jeder Hauptaufgabe:

```
git add .
git commit -m "task X - slytherin refactor"
git push
```

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 1 – Paradigmen Analyse

1.1

Erkläre den Unterschied zwischen:

imperativ
objektorientiert
funktional

Schreibe je **2 Sätze**.

1.2

Ordne folgende Probleme einem Paradigma zu:

GUI Anwendung
Datenpipeline
Spiel

1.3

Erkläre warum **OOP für große Systeme sinnvoll ist**.

1.4

Finde ein Beispiel aus der realen Welt für OOP.

📚 Nachschauen

https://www.geeksforgeeks.org/programming-paradigms/

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 2 – OOP Analyse

Gegeben:

```
class Dragon
{
   public string name;
   public int danger;
}
```

2.1

Identifiziere OOP-Probleme.

2.2

Verbessere Klasse mit Properties.

2.3

Füge Konstruktor hinzu.

2.4

Füge Methode hinzu:

PrintInfo()

📚

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/classes

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 3 – Encapsulation Refactor

Gegeben:

```
class Basilisk
{
   public int dangerLevel;
}
```

3.1

Warum ist das schlecht?

3.2

Refactor mit private Feld.

3.3

Erstelle Getter/Setter.

3.4

Validiere DangerLevel (1–10).

📚

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/properties

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 4 – Vererbung Design

Designfrage:

Hogwarts hat Kreaturen:

Dragon
Spider
Phoenix

4.1

Erstelle Basisklasse:

Creature

4.2

Definiere gemeinsame Eigenschaften.

4.3

Erstelle Unterklassen.

4.4

Implementiere Polymorphie Methode:

MakeSound()

📚

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 5 – Interface Design

Slytherin entwickelt ein Bewertungssystem.

5.1

Erstelle Interface:

IRateable

5.2

Methoden:

GetDangerLevel()
IsForbidden()

5.3

Implementiere Interface in zwei Klassen.

5.4

Nutze Interface in Liste.

📚

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/interfaces/

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 6 – SOLID Analyse

6.1

Erkläre **Single Responsibility Principle**.

6.2

Erkläre **Open Closed Principle**.

6.3

Erkläre **Dependency Inversion**.

6.4

Finde ein Beispiel aus deinem Code.

📚

https://www.freecodecamp.org/news/solid-principles-explained-in-plain-english/

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 7 – God Class Refactoring

Gegeben:

CreatureManager

enthält:

CreateCreature
PrintCreature
Statistics
Validation
Database

7.1

Warum ist das eine God Class?

7.2

Teile Klasse in Services auf.

7.3

Verschiebe Methoden.

7.4

Erkläre Vorteile.

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 8 – Service Layer Design

8.1

Erstelle Klasse:

CreatureService

8.2

Methoden:

AddCreature
GetCreature
RemoveCreature

8.3

Warum gehört diese Logik nicht ins Model?

8.4

Erkläre Vorteil von Service Layer.

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 9 – Repository Pattern

9.1

Erstelle Interface:

ICreatureRepository

9.2

Methoden:

Add
GetAll
GetByName

9.3

Implementiere Repository.

9.4

Verbinde Repository mit Service.

📚

https://martinfowler.com/eaaCatalog/repository.html

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 10 – LINQ Analyse

Liste:

List<Creature>

10.1

Berechne Durchschnitt DangerLevel.

10.2

Finde gefährlichste Kreatur.

10.3

Filter DangerLevel > 7.

10.4

Sortiere nach Name.

📚

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 11 – Exception Design

11.1

Wann sollte man Exceptions verwenden?

11.2

Erstelle eigene Exception:

DangerLevelException

11.3

Werfe Exception wenn Level > 10.

11.4

Fange Fehler im Main.

📚

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 12 – Architektur Analyse

Projektstruktur:

Models
Services
Repositories

12.1

Warum ist diese Struktur sinnvoll?

12.2

Welche Klasse gehört wohin?

12.3

Erstelle Beispielstruktur.

12.4

Erkläre Vorteile.

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 13 – Coupling & Cohesion

13.1

Erkläre Kopplung.

13.2

Erkläre Kohäsion.

13.3

Finde Beispiel in deinem Code.

13.4

Verbessere Design.

📚

https://refactoring.guru/design-patterns

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 14 – Code Smells

14.1

Definiere „Code Smell“.

14.2

Beispiele:

God Class
Duplicate Code
Long Method

14.3

Finde 2 Beispiele.

14.4

Refactore sie.

📚

https://refactoring.guru/refactoring/smells

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Aufgabe 15 – Architektur Review

15.1

Analysiere dein Projekt.

15.2

Identifiziere Verbesserung.

15.3

Refactore Struktur.

15.4

Erstelle Diagramm (optional).

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

TEIL 2

Abschlussprojekt

„Slytherin Magical Intelligence System“

Story

Das Haus Slytherin entwickelt für Hogwarts ein internes System zur **Analyse, Klassifizierung und Verwaltung magischer Wesen und Vorfälle**.

Das System soll nicht nur Daten speichern, sondern auch:

- Risiken bewerten
- Kategorien zuordnen
- Berichte erzeugen
- unterschiedliche Bewertungsstrategien unterstützen
- sauber aufgebaut und erweiterbar sein

Das ist also **kein Anfänger-CRUD**, sondern ein **kleines Architekturprojekt**.

Ziel des Projekts

Ihr baut ein System zur Verwaltung von:

- **Creatures**
- **Incidents**
- **Researchers**
- **Risk Reports**

Das System soll später leicht erweiterbar sein.

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Technische Ziele

Die Schüler sollen dabei anwenden:

- OOP
- SOLID
- Interfaces
- Repository Pattern
- Service Layer
- Strategy Pattern
- Factory Pattern
- Exception Handling
- LINQ
- Git Branch Workflow

Arbeitsform

Die Gruppe arbeitet **gemeinsam an einem großen Projekt**.

Empfehlung:

- 1–2 Schüler: Models + Interfaces
- 1–2 Schüler: Repository + Services
- 1–2 Schüler: Menü + Reports + Testing
- 1 Schüler: Git/Integration/Struktur im Blick

Git-Regeln

Nicht alles in `main`.

Branches

Beispiel:

```
git checkout -b feature/models
git checkout -b feature/repository
git checkout -b feature/services
git checkout -b feature/reports
```

Nach sinnvollen Zwischenschritten:

```
git add .
git commit -m "add creature model"
```

```
git push
```

Dann zusammenführen.

Projektstruktur

```
SlytherinMagicalIntelligenceSystem
│
├── Models
│   ├── Creature.cs
│   ├── Researcher.cs
│   ├── Incident.cs
│   └── RiskReport.cs
│
├── Interfaces
│   ├── IRepository.cs
│   ├── ICreatureRepository.cs
│   ├── IRiskStrategy.cs
│   └── IReportGenerator.cs
│
├── Repositories
│   ├── CreatureRepository.cs
│   ├── IncidentRepository.cs
│   └── ResearcherRepository.cs
│
├── Services
│   ├── CreatureService.cs
│   ├── IncidentService.cs
│   ├── RiskAnalysisService.cs
│   └── ReportService.cs
│
├── Factories
│   └── CreatureFactory.cs
│
├── Strategies
│   ├── StandardRiskStrategy.cs
│   ├── StrictRiskStrategy.cs
│   └── ResearchRiskStrategy.cs
│
├── Exceptions
│   ├── InvalidDangerLevelException.cs
│   └── InvalidIncidentException.cs
│
└── Program.cs
```

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Fachliche Idee

1. Creature

Magische Kreatur, z. B.:

- Basilisk
- Acromantula
- Dragon
- Thestral
- Hippogriff

2. Researcher

Slytherin-Schüler oder Hogwarts-Forscher, die Kreaturen untersuchen.

3. Incident

Ein Vorfall mit einer Kreatur, z. B.:

- escaped
- attacked student
- destroyed property
- magical disturbance

4. RiskReport

Zusammenfassung einer Bewertung:

- welche Kreatur
- welcher Incident
- welcher Risikowert
- welche Strategie
- Empfehlung

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Kernanforderungen

A. Models erstellen

1. Creature

Eigenschaften:

- Id
- Name
- Species
- DangerLevel
- Habitat
- IsRestricted
- CreatureType

Methoden:

- PrintInfo()
- Validate()

---

2. Researcher

Eigenschaften:

- Id
- Name
- House
- Specialization
- ExperienceLevel

Methoden:

- PrintInfo()

---

3. Incident

Eigenschaften:

- Id
- Title

- Description
- Severity
- Date
- CreatureId

Methoden:

- PrintSummary()

---

4. RiskReport

Eigenschaften:

- CreatureName
- IncidentTitle
- StrategyName
- RiskScore
- Recommendation

Methoden:

- PrintReport()

B. Interfaces

1. Generic Repository

```
public interface IRepository<T>
{
   void Add(T item);
   List<T> GetAll();
   T? GetById(int id);
   void Remove(int id);
}
```

---

2. ICreatureRepository

Erweitert spezielle Suche:

```
Creature? GetByName(string name);
List<Creature> GetRestrictedCreatures();
```

---

3. IRiskStrategy

```
public interface IRiskStrategy
{
   string Name { get; }
   int CalculateRisk(Creature creature, Incident incident);
   string GetRecommendation(int riskScore);
}
```

---

4. IReportGenerator

```
public interface IReportGenerator
{
   RiskReport Generate(Creature creature, Incident incident, IRiskStrategy strategy);
}
```

C. Repository Pattern

Erstellt In-Memory-Repositories.

CreatureRepository

Speichert List<Creature>

IncidentRepository

Speichert List<Incident>

ResearcherRepository

Speichert List<Researcher>

Wichtig:

- Add
- GetAll
- GetById
- Remove
- spezielle Suchmethoden

D. Service Layer

1. CreatureService

Verantwortlich für:

- Creature hinzufügen
- Validierung
- Restricted Creatures anzeigen
- Suche nach Name
- Statistik

Methodenideen:

- AddCreature(Creature creature)
- GetAllCreatures()
- GetMostDangerousCreature()
- GetAverageDangerLevel()
- GetRestrictedCreatures()

---

2. IncidentService

Verantwortlich für:

- Incident hinzufügen
- Vorfälle pro Kreatur anzeigen
- schwere Vorfälle filtern

Methodenideen:

- AddIncident(Incident incident)
- GetIncidentsForCreature(int creatureId)
- GetCriticalIncidents()

---

3. RiskAnalysisService

Verantwortlich für:

- Strategie auswählen
- Risiko berechnen
- Reports erzeugen

Methodenideen:

- CreateRiskReport(int creatureId, int incidentId, IRiskStrategy strategy)

---

4. ReportService

Verantwortlich für:

- Reports sammeln
- Reports anzeigen
- Reports filtern
- einfache Statistiken

E. Strategy Pattern

Das ist einer der Hauptpunkte.

Es soll **mehrere Bewertungsstrategien** geben.

1. StandardRiskStrategy

Berechnung z. B.:

RiskScore = DangerLevel * 10 + Severity * 5

2. StrictRiskStrategy

Gewichtet härter:

RiskScore = DangerLevel * 15 + Severity * 10

Wenn IsRestricted == true, dann Bonus auf RiskScore.

3. ResearchRiskStrategy

Etwas milder, weil Forschungskontext:

RiskScore = DangerLevel * 8 + Severity * 4

Empfehlung je nach Score

Beispiel:

- 0–30 → Monitor
- 31–60 → Caution
- 61–100 → Restricted Access
- 100 → Immediate Intervention

F. Factory Pattern

Für die Erstellung von Standardkreaturen:

CreatureFactory

Methodenideen:

- CreateDragon(string name)
- CreateBasilisk(string name)
- CreateHippogriff(string name)
- CreateAcromantula(string name)

Vorteil: Schüler sehen, wann Factory sinnvoll ist.

G. Exception Handling

Eigene Exceptions:

InvalidDangerLevelException

Wenn DangerLevel nicht 1–10 ist

InvalidIncidentException

Wenn Severity ungültig ist oder Creature nicht existiert

Die Services sollen validieren und Exceptions werfen.

Im Program.cs werden Fehler sauber abgefangen.

H. LINQ-Pflichtteile

Die Gruppe soll LINQ sinnvoll einsetzen für:

- durchschnittlicher DangerLevel
- gefährlichste Kreatur
- alle restricted creatures
- alle Incidents mit Severity > X
- Top 3 gefährlichste Kreaturen
- Incident-Anzahl pro Kreatur
- Reports mit hohem Risiko

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

I. Menüsystem

Das System soll ein sinnvolles Konsolenmenü haben.

Hauptmenü

1 - Add Creature
2 - Show Creatures
3 - Add Researcher
4 - Add Incident
5 - Show Incidents
6 - Generate Risk Report
7 - Show Reports
8 - Show Statistics
9 - Exit

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

Konkrete Projektaufgaben

Phase 1 – Grundstruktur

Aufgabe 1

Projekt anlegen, Ordnerstruktur bauen, Git initialisieren.

Aufgabe 2

Models anlegen.

Aufgabe 3

Enums anlegen:

- CreatureType
- House
- IncidentSeverity

Aufgabe 4

PrintInfo() / PrintSummary() Methoden einbauen.

Phase 2 – Architektur

Aufgabe 5

Generic Repository Interface bauen.

Aufgabe 6

CreatureRepository implementieren.

Aufgabe 7

IncidentRepository implementieren.

Aufgabe 8

ResearcherRepository implementieren.

Aufgabe 9

CreatureService bauen.

Aufgabe 10

IncidentService bauen.

Phase 3 – Risikoanalyse

Aufgabe 11

IRiskStrategy definieren.

Aufgabe 12

StandardRiskStrategy implementieren.

Aufgabe 13

StrictRiskStrategy implementieren.

Aufgabe 14

ResearchRiskStrategy implementieren.

Aufgabe 15

RiskAnalysisService bauen.

Aufgabe 16

RiskReport Model bauen.

Aufgabe 17

IReportGenerator + ReportService bauen.

Phase 4 – Factory + Validation

Aufgabe 18

CreatureFactory bauen.

Aufgabe 19

Custom Exceptions bauen.

Aufgabe 20

Validierungen in Services einbauen.

Phase 5 – Menü + Integration

Aufgabe 21

Hauptmenü bauen.

Aufgabe 22

Creature anlegen über Benutzereingabe.

Aufgabe 23

Incident anlegen über Benutzereingabe.

Aufgabe 24

Risk Report generieren über Menü.

Aufgabe 25

Statistikmenü bauen.

Phase 6 – Auswertung

Aufgabe 26

Top 3 gefährlichste Kreaturen.

Aufgabe 27

Kreatur mit meisten Vorfällen.

Aufgabe 28

Durchschnittliche Incident-Severity.

Aufgabe 29

Reports nach Risiko sortieren.

Aufgabe 30

Restricted Creatures extra anzeigen.

Zusatzaufgaben

Bonus 1 – Speicherung

Optional JSON speichern/laden.

Bonus 2 – Logging

Fehler und Aktionen in Textdatei protokollieren.

Bonus 3 – Simple Unit Tests

Für Risk Strategies und Services.

Bonus 4 – Report Export

Textdatei mit Report erzeugen.

Bonus 5 – Zweite Factory

Factory für Standard-Incidents.

Nachschauquellen

Interfaces

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/interfaces/

Erstellt von Erblin Kastrati & Chatty, Qualidy
10.03.2026

LINQ

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/

Exceptions

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/

Refactoring / God Class

https://refactoring.guru/smells/god-object

SOLID

https://www.freecodecamp.org/news/solid-principles-explained-in-plain-english/

Ihr seid das Slytherin Magical Systems Lab.
Eure Aufgabe ist nicht nur, dass das Programm läuft — sondern dass es **gut aufgebaut, erweiterbar und wartbar** ist.
Ihr entwickelt also nicht nur Code, sondern eine kleine Architektur